

Errata

Added by Aimbots, Oct 2026

The original document has some mistakes and contradictions. They are corrected below. **Error 1 is the one that will affect your assignment.**

Errors

1. **S1 and S2 offsets are swapped in Table 1 (p.3).** The right switch is S2, at bits 44–45, and the left switch is S1, at bits 46–47. The figure on p.3 (S1 on the left, S2 on the right) and the reference code on p.8 both agree, and so did our hardware test on a Type C board. In the code, S1 is commented “Switch left” and read from bits 46–47, and S2 is “Switch right” from bits 44–45. Only the table is wrong.
2. **Mouse button offsets in Table 2 (p.3) are wrong.** Left button = bit 96 (byte 12), not 86. Right button = bit 104 (byte 13), not 94. The struct’s byte comments are correct.
3. **The keyboard offset in Table 3 (p.4) is wrong.** Keys start at bit 112 (bytes 14–15), not 102.

Notes (not errors)

1. The red inverter note (p.3) is already handled by the Type C board’s hardware, so there’s nothing to do in code.



In the RoboMasters competition, team members must use the official DJI 2.4 GHz remote control system, specifically the DT7 and DR16. Details are as follows:

Official product page	http://www.dji.com/cn/product/dt7-dr16-rc-system
Remote controller driver	http://download.dji-innovations.com/downloads/dt7/cn/DT7&DR16_RC_System_User_Manual_v1.00_cn.pdf
User manual	http://download.dji-innovations.com/downloads/driver/DJI_WIN_Driver_Installer.exe

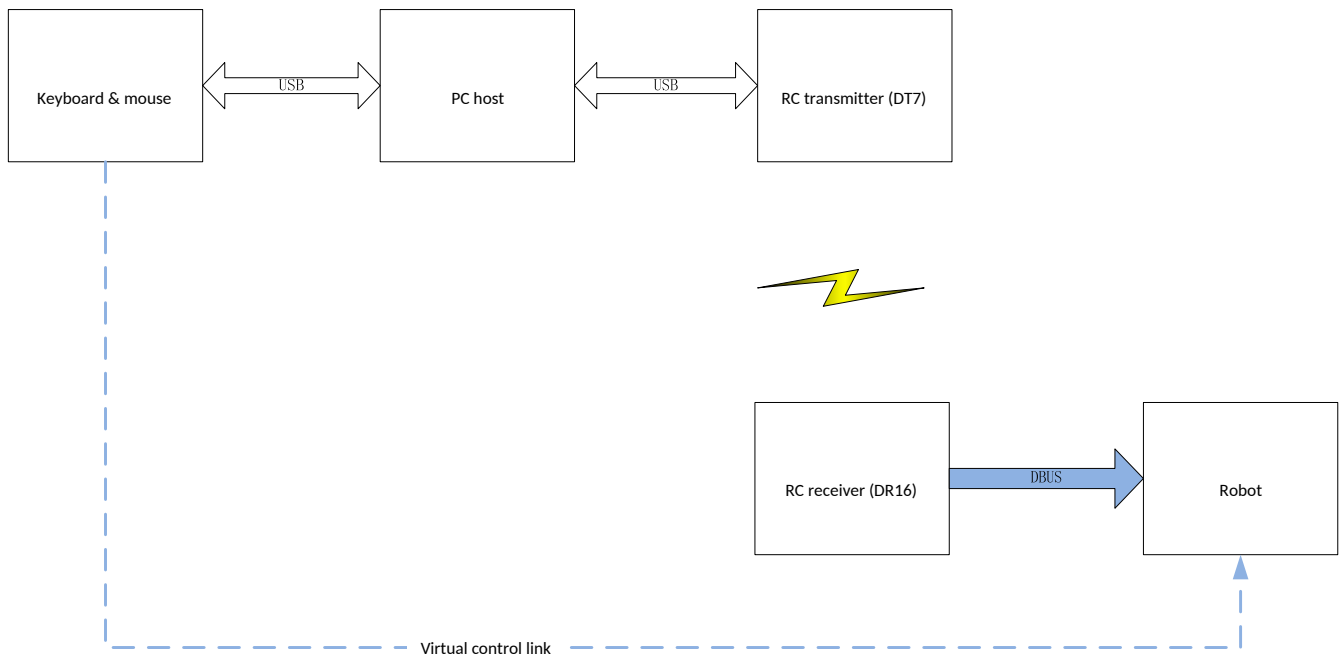
Special notes:

1: Install only the DT7 driver software; do not install the DT7&DR16 configuration software

2: The remote control system provided by RoboMasters is specially customized by DJI. Do not upgrade the remote controller firmware arbitrarily, or the remote controller will become unusable

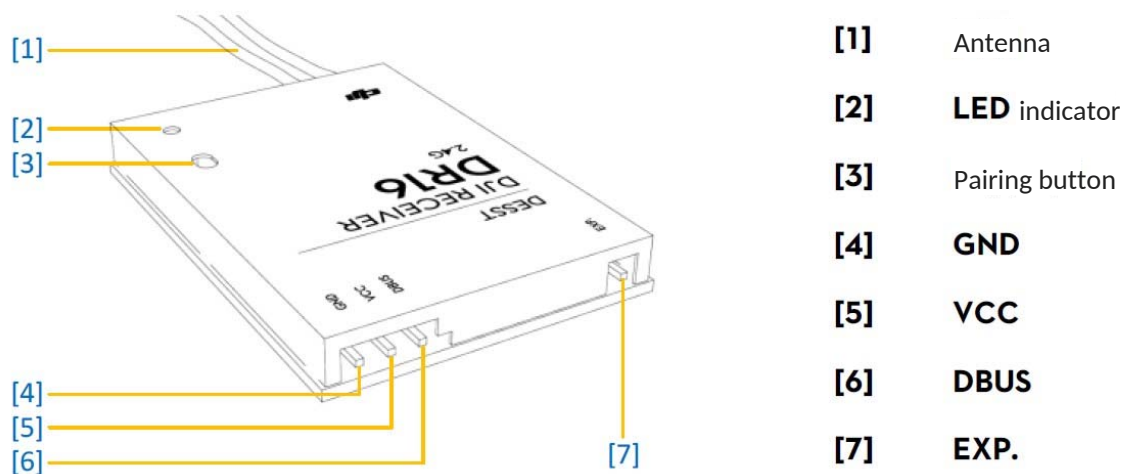
The receiver outputs standard DBUS protocol data. In addition, to make PC control of the robots easier, RoboMasters officially

added extra keyboard and mouse control information on top of the original remote controller design. The control link is shown below:



Next, this document focuses on the blue one-way DBUS control frame in the figure.

Remote controller receiver



Focus here on control wires 4, 5 and 6: 4 is ground; 5 is VCC, which powers the receiver, with a voltage range of 4–8.4 V; and 6 is the DBUS data output line.

Once the receiver and transmitter have established a connection, the receiver sends one frame of data (18 bytes) over DBUS every 7 ms. The DBUS communication parameters are as follows:

DBUS parameter	Value
Baud rate	100Kbps
Data bits	8
Parity	Even parity
Stop bits	1
Flow control	None

Note: The DBUS signal levels are TTL-compatible, but inverted relative to a normal UART signal, so an inverting transistor circuit must be added on the MCU side for the MCU to correctly recognize the UART signal.

The structure of the control frame is as follows:

Table 1 Remote controller information

Field	Channel 0	Channel 1	Channel 2	Channel 3	S1	S2
Offset	0	11	22	33	44	46
Length (bit)	11	11	11	11	2	2
Signed	No	No	No	No	No	No
Range	Max 1684 Mid 1024 Min 364	Max 1684 Mid 1024 Min 364	Max 1684 Mid 1024 Min 364	Max 1684 Mid 1024 Min 364	Max 3 Min 1	Max 3 Min 1
Function	Unsigned type Remote controller channel 0 control information	Unsigned type Remote controller channel 1 control information	Unsigned type Remote controller channel 2 control information	Unsigned type Remote controller channel 3 control information	Remote controller transmitter S1 switch position 1: Up 2: Down 3: Middle	Remote controller transmitter S2 switch position 1: Up 2: Down 3: Middle

The remote controller's channels and control switches are shown below:

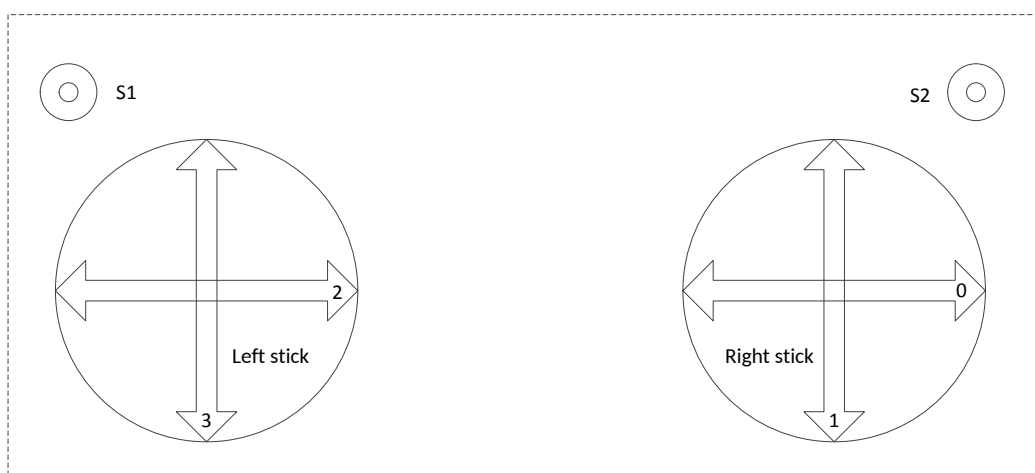


Table 2 Mouse information

Field	Mouse X axis	Mouse Y axis	Mouse Z axis	Left mouse button	Right mouse button
Offset	48	64	80	86	94
Length	16	16	16	8	8

Signed	Yes	Yes	Yes	No	No
Range	Max 32767 Min -32768 Rest value 0	Max 32767 Min -32768 Rest value 0	Max 32767 Min -32768 Rest value 0	Max 1 Min 0	Max 1 Min 0
Function	Mouse movement speed on X axis Negative: moving left Positive: moving right	Mouse movement speed on Y axis Negative: moving left Positive: moving right	Mouse movement speed on Z axis Negative: moving left Positive: moving right	Whether left button is pressed 0 --- not pressed 1 --- pressed	Whether right button is pressed 0 --- not pressed 1 --- pressed

Table 3 Keyboard information

Field	Keys	Reserved field
Offset	102	
Length	16	
Signed	No	
Range	Bit flags	
Function	Each key corresponds to one bit, as follows: Bit0 ----- W key Bit1 ----- S key Bit2 ----- A key Bit3 ----- D key Bit4 ----- Q key Bit5 ----- E key Bit6 ----- Shift key Bit7 ----- Ctrl key	

The corresponding data frame structure is as follows:

```
#pragma pack(1)
typedef union
{
    struct
    {
        struct
        {
            uint8_t  ch0_h:8;           //!< Byte 0

            uint8_t  ch0_l:3;           //!< Byte 1
            uint8_t  ch1_h:5;

            uint8_t  ch1_l:6;           //!< Byte 2
            uint8_t  ch2_h:2;

            uint8_t  ch2_m:8;           //!< Byte 3

            uint8_t  ch2_l:1;           //!< Byte 4
            uint8_t  ch3_h:7;

            uint8_t  ch3_l:4;           //!< Byte 5
```

```

        uint8_t  s1:2;
        uint8_t  s2:2;
    }rc;

    struct
    {
        int16_t x;           //!< Byte 6-7
        int16_t y;           //!< Byte 8-9
        int16_t z;           //!< Byte 10-11
        uint8_t press_l;     //!< Byte 12
        uint8_t press_r;     //!< Byte 13
    }mouse;

    struct
    {
        uint16_t v;          //!< Byte 14-15
    }key;

    uint16_t resv;           //!< Byte 16-17
};
uint8_t buf[18];           //!< Union --> Byte<0-17>
}RC_Ctl_Define_t;

```

Appendix: Reference code (STM32F4 platform)

```
// #pragma pack(1)
// typedef union
// {
//     struct
//     {
//         struct
//         {
//             uint8_t  ch0_h:8;           //!< Byte 0
//
//             uint8_t  ch0_l:3;           //!< Byte 1
//             uint8_t  ch1_h:5;
//
//             uint8_t  ch1_l:6;           //!< Byte 2
//             uint8_t  ch2_h:2;
//
//             uint8_t  ch2_m:8;           //!< Byte 3
//
//             uint8_t  ch2_l:1;           //!< Byte 4
//             uint8_t  ch3_h:7;
//
//             uint8_t  ch3_l:4;           //!< Byte 5
//             uint8_t  s1:2;
//             uint8_t  s2:2;
//         }rc;
//
//         struct
//         {
//             int16_t x;                 //!< Byte 6-7
//             int16_t y;                 //!< Byte 8-9
//             int16_t z;                 //!< Byte 10-11
//             uint8_t press_l;           //!< Byte 12
//             uint8_t press_r;           //!< Byte 13
//         }mouse;
//
//         struct
//         {
//             uint16_t v;                 //!< Byte 14-15
//         }key;
//
//             uint16_t resv;                 //!< Byte 16-17
//         };
//         uint8_t buf[18];                 //!< Union --> Byte<0-17>
//     }RC_Ctl_Define_t;

/* ----- RC Channel Definition----- */
#define RC_CH_VALUE_MIN      ((uint16_t)364 )
#define RC_CH_VALUE_OFFSET   ((uint16_t)1024)
#define RC_CH_VALUE_MAX      ((uint16_t)1684)

/* ----- RC Switch Definition----- */
#define RC_SW_UP              ((uint16_t)1)
#define RC_SW_MID             ((uint16_t)3)
#define RC_SW_DOWN            ((uint16_t)2)

/* ----- PC Key Definition----- */
#define KEY_PRESSED_OFFSET_W   ((uint16_t)0x01<<0)
#define KEY_PRESSED_OFFSET_S   ((uint16_t)0x01<<1)
#define KEY_PRESSED_OFFSET_A   ((uint16_t)0x01<<2)
#define KEY_PRESSED_OFFSET_D   ((uint16_t)0x01<<3)
#define KEY_PRESSED_OFFSET_Q   ((uint16_t)0x01<<4)
#define KEY_PRESSED_OFFSET_E   ((uint16_t)0x01<<5)
#define KEY_PRESSED_OFFSET_SHIFT ((uint16_t)0x01<<6)
#define KEY_PRESSED_OFFSET_CTRL ((uint16_t)0x01<<7)

/* ----- Data Struct ----- */
typedef struct
{
```

```

struct
{
    uint16_t ch0;
    uint16_t ch1;
    uint16_t ch2;
    uint16_t ch3;
    uint8_t s1;
    uint8_t s2;
}rc;

struct
{
    int16_t x;
    int16_t y;
    int16_t z;
    uint8_t press_l;
    uint8_t press_r;
}mouse;

struct
{
    uint16_t v;
}key;
}RC_Ctl_t;

/* ----- Internal Data ----- */
volatile unsigned char sbus_rx_buffer[25];
static RC_Ctl_t RC_Ctl;

/* ----- Function Implements ----- */

/*****
 * @fn      RC_Init
 *
 * @brief   configure stm32 usart2 port
 *          - USART Parameters
 *          - 100Kbps
 *          - 8-N-1
 *          - DMA Mode
 *
 * @return  None.
 *
 * @note    This code is fully tested on STM32F405RGT6 Platform, You can port it
 *          to the other platform.
 */
void RC_Init(void)
{
    /* ----- Enable Module Clock Source ----- */
    RCC_AHB1PeriphClockCmd(RCC_AHB1Periph_GPIOA | RCC_AHB1Periph_DMA1, ENABLE);
    RCC_APB1PeriphClockCmd(RCC_APB1Periph_USART2, ENABLE);
    GPIO_PinAFConfig(GPIOA, GPIO_PinSource3, GPIO_AF_USART2);

    /* ----- Configure GPIO ----- */
    {
        GPIO_InitTypeDef gpio;
        USART_InitTypeDef usart2;

        gpio.GPIO_Pin   = GPIO_Pin_3 ;
        gpio.GPIO_Mode   = GPIO_Mode_AF;
        gpio.GPIO_OType  = GPIO_OType_PP;
        gpio.GPIO_Speed  = GPIO_Speed_100MHz;
        gpio.GPIO_PuPd   = GPIO_PuPd_NOPULL;
        GPIO_Init(GPIOA, &gpio);

        USART_DeInit(USART2);
        usart2.USART_BaudRate           = 100000;
        usart2.USART_WordLength         = USART_WordLength_8b;
        usart2.USART_StopBits           = USART_StopBits_1;
        usart2.USART_Parity              = USART_Parity_Even;
        usart2.USART_Mode                = USART_Mode_Rx;
    }
}

```



```

    usart2.USART_HardwareFlowControl = USART_HardwareFlowControl_None;
    USART_Init(USART2, &usart2);

    USART_Cmd(USART2, ENABLE);
    USART_DMACmd(USART2, USART_DMAReq_Rx, ENABLE);
}

/* ----- Configure NVIC ----- */
{
    NVIC_InitTypeDef nvic;

    nvic.NVIC_IRQChannel          = DMA1_Stream5_IRQn;
    nvic.NVIC_IRQChannelPreemptionPriority = 1;
    nvic.NVIC_IRQChannelSubPriority   = 1;
    nvic.NVIC_IRQChannelCmd          = ENABLE;
    NVIC_Init(&nvic);
}

/* ----- Configure DMA ----- */
{
    DMA_InitTypeDef dma;

    DMA_DeInit(DMA1_Stream5);
    dma.DMA_Channel          = DMA_Channel_4;
    dma.DMA_PeripheralBaseAddr = (uint32_t)&(USART2->DR);
    dma.DMA_Memory0BaseAddr   = (uint32_t)sbus_rx_buffer;
    dma.DMA_DIR               = DMA_DIR_PeripheralToMemory;
    dma.DMA_BufferSize        = 18;
    dma.DMA_PeripheralInc     = DMA_PeripheralInc_Disable;
    dma.DMA_MemoryInc         = DMA_MemoryInc_Enable;
    dma.DMA_PeripheralDataSize = DMA_PeripheralDataSize_Byte;
    dma.DMA_MemoryDataSize    = DMA_MemoryDataSize_Byte;
    dma.DMA_Mode              = DMA_Mode_Circular;
    dma.DMA_Priority          = DMA_Priority_VeryHigh;
    dma.DMA_FIFOMode          = DMA_FIFOMode_Disable;
    dma.DMA_FIFOThreshold     = DMA_FIFOThreshold_1QuarterFull;
    dma.DMA_MemoryBurst       = DMA_Mode_Normal;
    dma.DMA_PeripheralBurst   = DMA_PeripheralBurst_Single;
    DMA_Init(DMA1_Stream5, &dma);

    DMA_ITConfig(DMA1_Stream5, DMA_IT_TC, ENABLE);
    DMA_Cmd(DMA1_Stream5, ENABLE);
}
}

/*****
 * @fn      DMA1_Stream5_IRQHandler
 *
 * @brief    USART2 DMA ISR
 *
 * @return   None.
 *
 * @note     This code is fully tested on STM32F405RGT6 Platform, You can port it
 *           to the other platform.
 */
void DMA1_Stream5_IRQHandler(void)
{
    if(DMA_GetITStatus(DMA1_Stream5, DMA_IT_TCIF5))
    {
        DMA_ClearFlag(DMA1_Stream5, DMA_FLAG_TCIF5);
        DMA_ClearITPendingBit(DMA1_Stream5, DMA_IT_TCIF5);

        RC_Ctl.rc.ch0 = (sbus_rx_buffer[0] | (sbus_rx_buffer[1] << 8)) & 0x07ff;    //!< Channel 0
        RC_Ctl.rc.ch1 = ((sbus_rx_buffer[1] >> 3) | (sbus_rx_buffer[2] << 5)) & 0x07ff;    //!< Channel 1
        RC_Ctl.rc.ch2 = ((sbus_rx_buffer[2] >> 6) | (sbus_rx_buffer[3] << 2) |          //!< Channel 2
                        (sbus_rx_buffer[4] << 10)) & 0x07ff;
        RC_Ctl.rc.ch3 = ((sbus_rx_buffer[4] >> 1) | (sbus_rx_buffer[5] << 7)) & 0x07ff;    //!< Channel 3
        RC_Ctl.rc.s1 = ((sbus_rx_buffer[5] >> 4) & 0x000C) >> 2;                      //!< Switch left
        RC_Ctl.rc.s2 = ((sbus_rx_buffer[5] >> 4) & 0x0003);                          //!< Switch right
    }
}

```

```

RC_Ctl.mouse.x = sbus_rx_buffer[6] | (sbus_rx_buffer[7] << 8);          //!< Mouse X axis
RC_Ctl.mouse.y = sbus_rx_buffer[8] | (sbus_rx_buffer[9] << 8);          //!< Mouse Y axis
RC_Ctl.mouse.z = sbus_rx_buffer[10] | (sbus_rx_buffer[11] << 8);        //!< Mouse Z axis
RC_Ctl.mouse.press_l = sbus_rx_buffer[12];                               //!< Mouse Left Is Press ?
RC_Ctl.mouse.press_r = sbus_rx_buffer[13];                               //!< Mouse Right Is Press ?

RC_Ctl.key.v = sbus_rx_buffer[14] | (sbus_rx_buffer[15] << 8);          //!< KeyBoard value
}
}

```